

TKM Internship Project Strategy & Plan

CIXIO — SmartHub: AI-Powered Personal Workspace

Document Version: 1.0

Date: May 2026

Prepared by: CIXIO Engineering Team

Classification: Internal — Intern Programme Management

Table of Contents

1. Executive Summary
 2. Intern Group Overview
 3. Project Concept: SmartHub
 4. Why This Project Fits CIXIO
 5. Project Modules — Full Breakdown
 6. Role Assignment by Group & Domain
 7. Cloud Infrastructure CIXIO Provides
 8. Technology Stack (Approved)
 9. Timeline & Milestones
 10. Deliverables per Group
 11. Evaluation Criteria
 12. Post-Internship Integration Plan
 13. Rules, Boundaries & Data Privacy
 14. Group-Specific Guidance
-

1. Executive Summary

What are we building?

Each of the 8 intern groups will independently build a complete, production-grade application called **SmartHub** — an AI-powered personal workspace assistant. Every group builds the same project in their own version, using their own GitHub accounts, their own machines, and without access to CIXIO's internal codebase or infrastructure.

Why this approach?

Problem

Solution

Can't share internal CIXIO code (security/IP)	Interns build from scratch — no shared codebase
Data privacy concerns with interns on CIXIO accounts	Interns use their own GitHub & email — zero exposure to CIXIO secrets
Need real output that CIXIO can use	Project mirrors CIXIO's actual platform stack — code is potentially reusable
Mixed skill levels across 8 groups	Same project, calibrated expectations — G1 builds more, G8 builds core modules
Interns need production-quality experience	Full-stack project with CI/CD, Docker, cloud deploy, real AI integration

What CIXIO gets at the end

- **8 independent versions** of a real AI-powered app — compare approaches, cherry-pick best implementations
- **Flutter mobile app code** that can supplement `cixio-mobile-app`
- **n8n automation workflows** that can be directly imported into CIXIO's n8n instance
- **RAG / LLM integration patterns** — new ideas for `cixio-ai` and `cixio-llm`
- **React/Next.js frontend patterns** — compare against `cixio-mvp-claude-webapp`
- **Docker + GitHub Actions CI/CD templates** — reusable pipeline configurations
- **Talent pipeline** — identify which interns are ready for full-time roles

2. Intern Group Overview

Group Composition Summary

Group	Label	Description	Students	Key Domains
G1	ALPHA	Elite — Startup-Ready	6	DevOps, Backend (×3), Frontend, Mobile
G2	BETA	Strong — High Potential	7	Backend (×3), AI/LLM (×2), DevOps, Frontend
G3	GAMMA	Mid-Upper — Developing Fast	7	Backend (×3), AI/LLM, DevOps, Frontend

G4	DELTA	Mid — Needs Guidance	7	Backend (×2), Frontend (×3), DevOps, AI/LLM
G5	EPSILON	Mid — Guided Learning	7	Backend (×2), AI/LLM (×3), Frontend (×2)
G6	ZETA	Mid-Low — Fundamentals Focus	7	Backend (×4), Mobile, Frontend (×2)
G7	ETA	Building — Foundation Work	7	Backend (×3), Frontend (×3), AI/LLM
G8	THETA	Entry — Intensive Mentoring	6	Frontend (×3), Backend (×2), AI/LLM

Total: 54 students

Student Domain Breakdown (All Groups)

Domain	Count	Modules They Own
Backend/API	22	Module 1 (Backend API)
Frontend/Web	13	Module 2 (Web App)
AI/LLM	10	Module 4 (AI Engine)
Infrastructure/DevOps	5	Module 6 (DevOps/CI-CD)
Mobile	3	Module 3 (Mobile App)

Note: Students without a dedicated match (e.g., only 1 mobile student in some groups) rotate into other modules as junior contributors. Groups with no DevOps student: DevOps module is assigned to the backend student with highest GitHub activity.

3. Project Concept: SmartHub

Project Name: **SmartHub — AI-Powered Personal Workspace**

SmartHub is a standalone, full-stack application where users can:

- 1. **Create and manage their workspace** — notes, tasks, documents, personal knowledge base
- 2. **Chat with AI** — using a local or cloud LLM (Ollama, OpenAI, Gemini free tier)
- 3. **Upload documents and ask questions** — RAG (Retrieval-Augmented Generation) — Q&A from their own files
- 4. **Receive smart notifications** — n8n-powered automation for reminders, email digests, and scheduled summaries
- 5. **Access everything from mobile** — Flutter app synced with the web app
- 6. **Monitor system health** — Grafana dashboard showing usage, API calls, latency (optional/advanced)

Why "SmartHub"?

It is the intern-safe, open-source analogue of what CIXIO already builds:

SmartHub Module	Maps to CIXIO Product
Backend API	<code>cixio-mvp-claude-backend</code> , <code>cixio-mobile-app-backend</code>
Web App (React)	<code>cixio-mvp-claude-webapp</code> , <code>landing1</code>
Mobile App (Flutter)	<code>cixio-mobile-app</code> , <code>CIXIO_LH_MOBILE_APP</code>
AI/LLM Engine	<code>cixio-llm</code> , <code>cixio-ai</code> (RAG platform)
n8n Automation	<code>cixio-notifications</code> , <code>cixio-workflows</code>
Docker + GitHub Actions	All CIXIO repos (same stack)
Monitoring (Grafana)	<code>cixio-monitoring</code>

4. Why This Project Fits CIXIO

Direct Integration Potential

What Interns Build	How CIXIO Uses It
Flutter mobile app	Direct code review → cherry-pick UI components, navigation patterns, API call structure for <code>cixio-mobile-app</code>

n8n workflow files	n8n workflow JSONs are importable — best group's workflows go directly into CIXIO's n8n
RAG pipeline	8 different approaches to document ingestion, embedding, and retrieval → benchmark against <code>cixio-ai</code>
FastAPI backend	API structure, auth patterns, SQLite/PostgreSQL schema design → inform next API versions
React/Next.js frontend	UI patterns, component design, state management approaches → compare with <code>cixio-mvp-claude-webapp</code>
GitHub Actions CI/CD	Templates → reuse across CIXIO repos (already using same stack)
Docker Compose files	Potentially reusable for new CIXIO microservices

Zero Risk to CIXIO

- No interns have access to CIXIO's AWS, GitHub Enterprise, or production systems
- No CIXIO source code is shared
- n8n, Jupyter, and Anaconda are deployed on a **separate, isolated cloud instance** from CIXIO's production
- Interns only see the public API endpoints of cloud tools (n8n webhook URLs, Jupyter URLs)

5. Project Modules — Full Breakdown

All 8 groups build **all 7 modules**. The group's skill level determines depth, not breadth.

Module 1: Backend API

Owner domain: Backend/API students

Technologies: Python (FastAPI) — recommended; Node.js (Express) — allowed

Minimum viable output: Working REST API with auth, CRUD for notes/tasks, SQLite storage

Core Features (all groups must build):

- User registration and login (JWT-based authentication)

- CRUD operations for: Notes, Tasks, Documents (metadata), User Profile
- File upload endpoint (store locally with metadata in DB)
- Health check endpoint (`GET /health`)
- OpenAPI/Swagger docs auto-generated

Advanced Features (G1–G3 expected; G4–G8 bonus):

- PostgreSQL instead of SQLite
- Pagination, filtering, search
- Rate limiting
- Background tasks (e.g., send welcome email via n8n webhook)
- Unit tests (pytest or Jest) — minimum 60% coverage

API Endpoints Required (minimum):

```
None
POST    /auth/register
POST    /auth/login
GET     /auth/me
GET     /notes          (list)
POST    /notes          (create)
GET     /notes/{id}   (get)
PUT     /notes/{id}   (update)
DELETE  /notes/{id}   (delete)
POST    /documents/upload
GET     /documents
GET     /health
GET     /docs          (Swagger UI)
```

Database Schema (minimum):

```
None
users (id, email, password_hash, name, created_at)
notes (id, user_id, title, content, tags, created_at, updated_at)
tasks (id, user_id, title, description, status, due_date,
created_at)
documents (id, user_id, filename, filepath, size, created_at)
```

Module 2: Web App (Frontend)

Owner domain: Frontend/Web students

Technologies: React (Vite) or Next.js — recommended; Vue 3 — allowed

Minimum viable output: Functional web app connected to Module 1 API

Core Features (all groups must build):

- Login / Register page
- Dashboard (notes list, tasks summary, recent documents)
- Notes: create, edit, delete with rich text or markdown
- Tasks: kanban board or list view with status toggle
- AI Chat: text input → backend → LLM response → display in chat UI
- Document upload: drag-and-drop + file list
- Responsive (works on mobile browser)

Advanced Features (G1–G3 expected; G4–G8 bonus):

- Dark mode toggle
- Real-time updates (WebSocket or polling)
- RAG Query UI: upload doc → ask question → show answer with source citations
- PWA with service worker (offline support)
- Search across notes

UI/UX Requirements:

- Clean, modern design (Tailwind CSS or Material UI or shadcn/ui)
 - Loading states for all async operations
 - Error handling with user-friendly messages
 - Mobile responsive
-

Module 3: Mobile App

Owner domain: Mobile students; all other students contribute as junior contributors

Technologies: Flutter (Dart) — required (aligns with CIXIO's stack)

Minimum viable output: Flutter app with login, notes CRUD, and AI chat screen

Core Features (all groups must build):

- Login / Register screen
- Home dashboard (notes list)
- Notes: view, create, edit, delete
- AI Chat screen (send message → API → display response)
- Basic settings screen (profile, logout)

Advanced Features (G1–G3 expected; G4–G8 bonus):

- Document upload from device camera/gallery
- Push notifications (via FCM)
- Offline mode with local SQLite sync
- Biometric login (fingerprint)
- RAG Query UI

Note for groups with no dedicated Mobile student:

The Backend student with the most GitHub activity leads Module 3. The mobile app must at minimum complete the core features. Seek help from CIXIO if stuck on Flutter setup.

Module 4: AI / LLM Engine

Owner domain: AI/LLM students

Technologies: Python (FastAPI, httpx, LangChain optional)

AI Backends (choose one):

- **Ollama** (local — best for understanding internals, no API cost)
- **OpenAI API** (free tier or students' own key — fastest to get working)
- **Google Gemini API** (free tier available)

Minimum viable output: A working `/v1/chat` endpoint + a working RAG query endpoint

Core Features (all groups must build):

a) Chat Completion API

None

POST `/ai/chat`

Body: { "messages": [{"role": "user", "content": "..."}],
"model": "..."} }

Response: { "reply": "...", "model": "...", "tokens_used": 123 }

- Accepts conversation history
- Configurable system prompt
- Streams response (SSE) — bonus

b) RAG — Document Q&A

None

POST `/ai/rag/ingest`

– Upload and embed a document

POST `/ai/rag/query`

– Ask a question about ingested documents

GET `/ai/rag/documents`

– List ingested documents

```
DELETE /ai/rag/documents/{id} - Remove document
```

RAG Implementation (minimum):

1. Accept PDF, .txt, .md files
2. Split into chunks (500–1500 chars with overlap)
3. Generate embeddings (use `nomic-embed-text` via Ollama, or OpenAI `text-embedding-3-small`)
4. Store in vector database (ChromaDB — recommended; FAISS — allowed)
5. On query: embed question → similarity search → top-k chunks → prompt LLM → return answer with citations

Advanced Features (G1–G3 expected; G4–G8 bonus):

- Multi-document RAG (multiple files per workspace)
- Reranker (rerank retrieved chunks before generation)
- Conversation memory (multi-turn RAG)
- Token usage logging + cost estimation
- Support Word documents (`.docx`) and PowerPoint (`.pptx`)

Models to Use:

Use Case	Local (Ollama)	Cloud (Free)
Chat	llama3.2:3b or phi3:mini	Gemini Flash, GPT-4o-mini
Embedding s	nomic-embed-text	OpenAI text-embedding-3-small
RAG answer	llama3.2:3b	Gemini Flash

Module 5: Automation (n8n)

Owner domain: Assigned to any remaining students; can be the AI/LLM or DevOps student as secondary

Technologies: n8n (visual workflow automation)

Platform: CIXIO provides a pre-deployed n8n instance. Interns are given credentials to configure workflows only. No cloud access is given.

Minimum viable output: 3 working n8n workflows exported as JSON

Required Workflows (all groups):

Workflow 1 — Welcome Email

- Trigger: Webhook called from SmartHub backend after registration
- Action: Send welcome email to new user with their name
- Tools: HTTP Request node → Email (SMTP or Gmail)

Workflow 2 — Daily Summary Digest

- Trigger: Schedule (cron) — 9:00 AM every day
- Action: Call SmartHub API → get user's tasks due today → format email → send
- Tools: Schedule Trigger → HTTP Request → Set → Email

Workflow 3 — AI Insight Report

- Trigger: Webhook or Schedule
- Action: Fetch recent notes from SmartHub → send to LLM → get a summary → email to user
- Tools: HTTP Request (notes API) → HTTP Request (AI chat API) → Email

Advanced Workflows (G1–G3 expected; G4–G8 bonus):

Workflow 4 — New Document Notification

- When a new document is uploaded to SmartHub, notify user via email with a document summary (call AI endpoint)

Workflow 5 — Smart Task Reminder

- Check for overdue tasks every hour → send push notification / email

Deliverable: Export all workflows as `.json` files and include in the project repository.

Module 6: DevOps / CI-CD / Cloud

Owner domain: Infrastructure/DevOps students (or highest GitHub-activity Backend student as fallback)

Technologies: Docker, Docker Compose, GitHub Actions

Minimum viable output: Docker Compose file that runs the entire stack; GitHub Actions CI pipeline

Required Infrastructure:

a) Docker Compose (`docker-compose.yml`) Must containerize the entire SmartHub stack:

None

```
services:
  backend:      # Module 1 – FastAPI or Node.js
  frontend:     # Module 2 – Nginx-served React build
  ai-engine:    # Module 4 – FastAPI AI service
  db:           # PostgreSQL (or SQLite via volume)
  ollama:       # (optional) local LLM backend
```

- All services connected on a private Docker network
- Environment variables via `.env` file
- Volumes for persistent data (database, uploaded files, Ollama models)
- Health checks on all services

b) GitHub Actions CI Pipeline (`.github/workflows/ci.yml`)

None

```
on: [push, pull_request]
jobs:
  lint-and-test:
    - Lint (Python: ruff/flake8; JS: eslint)
    - Unit tests (pytest / Jest)
    - Build Docker image (confirm it builds)
```

Advanced Features (G1–G3 expected; G4–G8 bonus):

- CD: Auto-deploy to a cloud free tier (Railway, Render, or Fly.io — free plans)
- Multi-stage Dockerfile (builder → production image, minimal size)
- GitHub Actions: build and push Docker image to GitHub Container Registry (GHCR)
- Secrets management: all credentials via GitHub Secrets / env vars, never hardcoded
- HTTPS via Let's Encrypt or cloud platform's built-in TLS

Module 7: Monitoring (Optional / Advanced)

For: G1 ALPHA, G2 BETA, G3 GAMMA — expected. All others: bonus.

Technologies: Prometheus, Grafana, or cloud platform equivalent

Minimum viable output: A Grafana dashboard with at least 4 panels

Core Dashboard Panels:

1. **API request rate** (requests/minute) — from Prometheus metrics endpoint on backend
2. **API error rate** (HTTP 4xx/5xx)
3. **AI chat usage** (total chats per day)

4. **System health** (CPU, memory via Prometheus node exporter)

Implementation:

- Add Prometheus metrics to FastAPI backend
(`prometheus-fastapi-instrumentator`)
 - Add `prometheus` and `grafana` to `docker-compose.yml`
 - Create Grafana dashboard JSON (exportable)
 - Optional: Loki for log aggregation
-

6. Role Assignment by Group & Domain

How role assignment works:

Each student is assigned a **primary module** based on their declared preferred domain. They contribute to that module as the lead contributor. They are expected to also understand and contribute (as junior) to at least one adjacent module.

Preferred Domain	Primary Module	Adjacent Module
Backend/API	Module 1 (Backend)	Module 6 (DevOps)
Frontend/Web	Module 2 (Web App)	Module 1 (Backend — API integration)
Mobile	Module 3 (Mobile App)	Module 2 (UI patterns)
AI/LLM	Module 4 (AI Engine)	Module 1 (Backend — AI endpoint integration)
Infrastructure/DevOps	Module 6 (DevOps)	Module 1 (Backend containerization)

Module 5 (n8n Automation) — Assignment Rule:

- If a group has an odd number of Backend students (e.g., 3 Backend students): the 3rd Backend student co-leads Module 5 with the AI student
 - If no DevOps student: Backend student with the most GitHub repos leads Module 6
-

Student	Domain	Module Lead	Secondary
Dany James	Infrastructure/DevOps	Module 6 (DevOps + CI/CD)	Module 7 (Monitoring)
Devna M Lal	Backend/API	Module 1 (Backend API)	Module 5 (n8n)
Aneena Fathima A R	Backend/API	Module 1 (Backend API)	Module 4 (AI integration)
Muhammed Asif A	Frontend/Web	Module 2 (Web App)	Module 3 (Mobile, junior)
Jayaharsh M	Mobile	Module 3 (Mobile App)	Module 2 (UI patterns)
Vedha	Backend/API	Module 4 (AI Engine lead) + Module 5	Module 1 (support)

G1 expected to complete all 7 modules including Monitoring. Aim for production deployment on cloud.

G2 · BETA — Role Assignments

Student	Domain	Module Lead	Secondary
Albin P Shaji	Backend/API	Module 1 (Backend API)	Module 6 (Docker)
S K Sneha	Backend/API	Module 1 (API + DB schema)	Module 5 (n8n)
ABEL TOM AUGUSTINE	AI/LLM	Module 4 (AI Engine — lead)	Module 5 (n8n)
SARFARAS YOONAS UTHINATTU	AI/LLM	Module 4 (RAG pipeline)	Module 1 (AI endpoint)
ABHINAND LAL	Infrastructure/DevOps	Module 6 (DevOps + CI/CD)	Module 7 (Monitoring)

Khushi	Frontend/Web	Module 2 (Web App)	Module 3 (Mobile, junior)
Aadithya P K	Backend/API	Module 3 (Mobile App lead)	Module 1 (API support)

G2: Full completion expected. Monitoring is a target. Aim for cloud deployment.

G3 · GAMMA — Role Assignments

Student	Domain	Module Lead	Secondary
Aayisha N	Backend/API	Module 1 (Backend API)	Module 5 (n8n)
Neeta A Suresh	Backend/API	Module 1 (DB + auth)	Module 6 (Docker)
Amaljith S J	AI/LLM	Module 4 (AI Engine)	Module 5 (n8n AI workflow)
A V Kiran	Backend/API	Module 3 (Mobile App lead)	Module 1 (API support)
Raehaan IH	Infrastructure/DevOps	Module 6 (DevOps + CI/CD)	Module 7 (Monitoring)
Dhrupan D	Frontend/Web	Module 2 (Web App)	Module 3 (Mobile, junior)
Nandana Manoj	Backend/API	Module 4 (RAG) + Module 5	Module 1 (support)

G3: Full completion expected. Monitoring is bonus. Aim for Docker Compose running locally at minimum.

G4 · DELTA — Role Assignments

Student	Domain	Module Lead	Secondary
---------	--------	-------------	-----------

Sahal Muzammil	Backend/API	Module 1 (Backend API lead)	Module 6 (Docker)
Sanas S	Frontend/Web	Module 2 (Web App)	Module 5 (n8n webhook UI)
Achuth B Nair	Frontend/Web	Module 2 (UI components)	Module 3 (Mobile, junior)
Akarsh S S	Infrastructure/DevOps	Module 6 (DevOps)	Module 1 (Docker)
Amith A	AI/LLM	Module 4 (AI Engine)	Module 5 (n8n)
Akshith K	Frontend/Web	Module 3 (Mobile App lead)	Module 2 (UI)
Aleena C K	Backend/API	Module 1 (DB + CRUD)	Module 4 (AI integration)

G4: Core modules (1, 2, 3, 4, 5, 6) expected. Monitoring optional. Focus on quality over quantity.

G5 · EPSILON — Role Assignments

Student	Domain	Module Lead	Secondary
Mohammed Fawaz	Backend/API	Module 1 (Backend API lead)	Module 6 (Docker)
Akshay T Geevarghese	AI/LLM	Module 4 (AI Engine lead)	Module 5 (n8n)
Nived Krishna S	Frontend/Web	Module 2 (Web App lead)	Module 3 (Mobile, junior)
Ajith Bhaskaran K S	AI/LLM	Module 4 (RAG pipeline)	Module 1 (AI endpoint)
Nezil Shah	AI/LLM	Module 4 (embeddings + vector DB)	Module 5 (n8n AI workflow)

Nisthula M	Frontend/We b	Module 3 (Mobile App lead)	Module 2 (UI)
Malavika M	Backend/API	Module 1 (auth + DB) + Module 6	Module 5 (n8n)

G5: Core modules expected. AI module has 3 students — divide into chat, RAG, and embeddings sub-tasks.

G6 · ZETA — Role Assignments

Student	Domain	Module Lead	Secondary
Mohammed Kutty PL	Backend/API	Module 1 (Backend API lead)	Module 5 (n8n)
Seljo E L	Backend/API	Module 1 (DB + CRUD)	Module 6 (Docker)
Aditya RP	Backend/API	Module 1 (auth + file upload)	Module 4 (AI endpoint integration)
Sruthin N	Mobile	Module 3 (Mobile App lead)	Module 2 (Mobile-web parity)
Ameen Anwar	Backend/API	Module 4 (AI Engine lead)	Module 5 (n8n)
Athila Zubair N V	Frontend/We b	Module 2 (Web App)	Module 3 (Mobile, junior)
Anandu A	Frontend/We b	Module 2 (UI components)	Module 5 (n8n webhook trigger)

G6: Focus on getting all 6 core modules (1–6) working and integrated. Quality of Backend and AI module is the priority.

G7 · ETA — Role Assignments

Student	Domain	Module Lead	Secondary
---------	--------	-------------	-----------

Adithyadev MR	Frontend/We b	Module 2 (Web App lead)	Module 5 (n8n webhook UI)
Hrideesh	AI/LLM	Module 4 (AI Engine lead)	Module 5 (n8n AI)
Anish Ramdas	Frontend/We b	Module 2 (UI components)	Module 3 (Mobile, junior)
Karthik S	Frontend/We b	Module 3 (Mobile App lead)	Module 2 (UI)
Amrah KS	Backend/API	Module 1 (Backend API lead)	Module 6 (Docker)
Mohammed Roshan P	Backend/API	Module 1 (auth + DB)	Module 5 (n8n)
Jasim Khader Ismail	Backend/API	Module 1 (CRUD + file upload)	Module 4 (AI endpoint)

G7: Prioritise Modules 1, 2, 4 first. Module 3 (Mobile) and Module 6 (DevOps) are secondary. Good team size for backend work.

G8 · THETA — Role Assignments (Intensive Mentoring)

Student	Domain	Module Lead	Secondary
Vaishnav Venugopal	Frontend/We b	Module 2 (Web App lead)	Module 5 (n8n)
Adithi S Nair	Frontend/We b	Module 2 (UI components)	Module 3 (basic Mobile)
Adithyan R	Backend/API	Module 1 (Backend API lead)	Module 6 (Docker basic)
Akshay B S	AI/LLM	Module 4 (AI Engine — basic chat only)	Module 5 (n8n)
Ajmal M S	Frontend/We b	Module 3 (Mobile App — basic)	Module 2 (UI)

Ahamad K Muhammed	Backend/API	Module 1 (auth + CRUD)	Module 4 (AI endpoint integration)
----------------------	-------------	------------------------	------------------------------------

G8: Focus on Modules 1, 2, and 4 (basic chat only). Module 3 should reach at least Login + Notes screens. Module 6: basic Docker Compose that runs backend. n8n: at least Workflow 1 (Welcome Email). CIXIO will provide extra check-ins for this group.

7. Cloud Infrastructure CIXIO Provides

Interns do **not** get access to CIXIO's production systems. CIXIO will deploy **separate, isolated instances** of developer tools on a cloud machine specifically for this internship. Interns are given **only the URL and login credentials** — no SSH, no root access, no cloud console access.

What CIXIO Deploys for Interns

Tool	Purpose	What Interns Get
n8n	Workflow automation	URL + login credentials. They build/test workflows, export as JSON
Jupyter Notebook	AI experimentation, data analysis	URL + login. They can run Python notebooks for RAG prototyping
Anaconda/JupyterHub	Python environment for AI/ML	URL + login. Conda environments available
Ollama endpoint (optional)	LLM inference for interns without powerful local machines	API base URL + key. They point their AI module here
Shared MongoDB (optional)	For groups needing persistent storage during demo	Connection string scoped to their group's database

What Interns Must Run Locally

Component	Why Local
Backend API	Their own code — runs on <code>localhost</code>

Web Frontend	<code>npm run dev</code> on their own machine
Mobile App	Flutter emulator or physical device
AI Engine	Run locally or use CIXIO's Ollama endpoint
Docker Compose	Local Docker Desktop or Docker Engine
GitHub Actions	Runs on GitHub's free runners (their own repos)

Important Rules for Cloud Tools

1. **n8n:** Do not store sensitive credentials in n8n credentials. Use placeholder values. CIXIO manages the actual SMTP configuration — interns only configure webhook URLs and logic.
2. **Jupyter:** Do not upload any CIXIO internal documents to Jupyter. Only use dummy/test data.
3. **Ollama:** If using CIXIO's shared Ollama endpoint, do not send any real personal data in prompts during development.
4. **No SSH access** will be granted. If a tool doesn't work, report to CIXIO contact.

8. Technology Stack (Approved)

Interns must use only **approved technologies** to ensure CIXIO can evaluate and potentially integrate their work.

Backend API

Option	Language	Framework	Database	Notes
 Recommended	Python 3.11+	FastAPI	SQLite (dev) / PostgreSQL (prod)	Aligns with CIXIO's stack
 Allowed	Node.js 20+	Express.js	SQLite / PostgreSQL	Accepted alternative
 Not allowed	PHP, Ruby, .NET	Any	Any	Not in CIXIO stack

Frontend Web App

Option	Framework	Styling	Notes
✓ Recommended	React 18 + Vite	Tailwind CSS	Most common in industry
✓ Allowed	Next.js 14	Tailwind / shadcn/ui	Fine for SSR
✓ Allowed	Vue 3 + Vite	Tailwind	Acceptable
✗ Not allowed	Angular, Svelte	Any	Not in CIXIO stack

Mobile App

Option	Framework	Notes
✓ Required	Flutter 3.x (Dart)	CIXIO's production mobile stack
✗ Not allowed	React Native, Kotlin, Swift	Won't align with cixio-mobile-app

AI / LLM Engine

Option	Notes
✓ Recommended	Python, FastAPI, Ollama SDK (httpx)
✓ Allowed	LangChain (Python) — for RAG pipeline if comfortable
✓ Allowed	OpenAI Python SDK (works with any OpenAI-compatible endpoint)
✓ Allowed	Google Generative AI SDK (Gemini free tier)
✓ Vector DB: ChromaDB	Recommended — easy local setup, Python native
✓ Vector DB: FAISS	Allowed — faster but lower-level
✗ Not recommended	Pinecone, Weaviate (paid) unless using free tier

Automation

Tool	Notes
✓ Required	n8n (use CIXIO's hosted instance)
✓ Also allowed	GitHub Actions (for automation of builds/tests)

DevOps

Tool	Notes
✓ Required	Docker + Docker Compose
✓ Required	GitHub Actions (.github/workflows/)
✓ Allowed	GitHub Container Registry (GHCR) for image hosting
✓ Allowed cloud deploy	Railway, Render, Fly.io (all have free tiers)

Monitoring (optional/advanced)

Tool	Notes
✓ Recommended	Prometheus + Grafana (add to docker-compose)
✓ Allowed	Grafana Cloud free tier

9. Timeline & Milestones

Total Duration: 8 weeks (adjust based on actual internship calendar)

Week-by-Week Plan

Week	Focus	Deliverable
------	-------	-------------

Week 1	Kickoff + Setup	Project repo created on GitHub (own account), README with architecture diagram, tech stack confirmed, local dev environment running, team roles assigned
Week 2	Backend Core	Module 1: Auth (register/login/JWT), basic CRUD for Notes. Module 6: Initial <code>Dockerfile</code> for backend
Week 3	Frontend + Mobile Core	Module 2: Login + Notes list page connected to API. Module 3: Flutter app running with Login screen. Module 4: AI chat endpoint (<code>/ai/chat</code>) working locally
Week 4	Integration Sprint 1	All modules talking to each other locally. Web app displays AI chat. Flutter app calls API. n8n Workflow 1 (Welcome Email) working. First <code>docker-compose.yml</code> (backend + frontend) running
Week 5	AI/RAG + Automation	Module 4: RAG pipeline working (ingest file → ask question). n8n Workflows 2 & 3 working. Module 3: Flutter Notes CRUD working
Week 6	Integration Sprint 2	Full stack running via Docker Compose. GitHub Actions CI passing. RAG Q&A visible in Web UI. Mobile app connects to all API endpoints. All 3 n8n workflows tested
Week 7	Polish + Advanced Features	Fix bugs. Add unit tests. Advanced features per group level. Optional: cloud deployment. Monitoring dashboard (G1–G3). Code review within group
Week 8	Final Demo + Submission	Final presentation. All deliverables submitted. Live demo of the full stack. Code freeze. Documentation completed

Weekly Check-ins with CIXIO

- **Every Monday:** 15-minute async update (text) — each group posts a brief progress update to their designated CIXIO contact (email or channel)
- **Week 4 (Mid-Point Review):** Live 30-minute demo per group — show what's working
- **Week 8 (Final Demo):** 45-minute live demo + Q&A per group
- **G8 THETA:** Additional weekly 30-min check-in call with CIXIO mentor (intensive track)

10. Deliverables per Group

Each group must submit the following by Week 8:

A. GitHub Repository

- Public repository on the group's GitHub (one repo per group, or a GitHub Organization created by the group)
- **Repository name pattern:** `smarthub-[group-label]` (e.g., `smarthub-alpha`, `smarthub-beta`)
- All modules in the same repo (monorepo) or separate repos with a parent README linking them
- Clean commit history — meaningful commit messages
- `.gitignore` for secrets, `node_modules`, build artifacts, `.env`
- **NO secrets committed** (no API keys, passwords, or tokens in any commit — ever)

B. README.md (Root)

Must include:

1. Project name and one-line description
2. Architecture diagram (can be drawn with Mermaid or any diagram tool)
3. Tech stack list
4. Prerequisites (what to install locally)
5. How to run locally (`docker-compose up` should work)
6. How to run without Docker (dev mode instructions)
7. API documentation link (Swagger URL)
8. Team member names and their module ownership
9. Demo video link (YouTube unlisted or Loom)

C. Working Application

- `docker-compose up` must start the full stack
- All core features must work (refer to Module descriptions above)
- No placeholder UIs — actual functionality required

D. n8n Workflow Files

- Export all n8n workflows as `.json` files
- Place in `/automation/` folder in the repo
- Include a README in `/automation/` explaining each workflow

E. Demo Video

- 5–10 minute screen recording walking through the full app
- Show: Login → Create Note → Upload Document → Ask AI about document → Check n8n workflow running
- Upload to YouTube (unlisted) or Loom

- Link in README

F. Architecture Diagram

- Shows how all modules connect
- Can be `diagram.png` or a Mermaid diagram in README
- Must show: user → web/mobile → backend → AI engine → vector DB → LLM

G. Presentation Deck (Week 8 Demo)

- 10–15 slides
- Slides: Overview, Architecture, Module demos, Challenges faced, What you'd improve, What you learned

11. Evaluation Criteria

Scoring Rubric (out of 100 points)

Category	Weight	What we look for
Functionality	30%	All core features work. App doesn't crash on basic use cases. API endpoints return correct responses
Code Quality	20%	Readable code, meaningful variable names, no duplication. Functions are small and focused. No hardcoded secrets
Integration	20%	All modules are connected and talking to each other. Docker Compose brings up the full stack
AI Implementation	15%	Chat works. RAG pipeline returns relevant answers with source citations. Embeddings are being used
Documentation	10%	README is complete, architecture diagram exists, demo video is clear
Bonus: Advanced Features	5%	Cloud deployment, monitoring dashboard, PWA, push notifications, etc.

Per-Group Calibration

Evaluation is **group-tier adjusted**:

- **G1 ALPHA:** Full marks require all 7 modules + cloud deployment + monitoring
- **G2–G3:** Full marks require all 6 core modules + cloud deployment
- **G4–G5:** Full marks require modules 1–5 + basic DevOps (Docker only)
- **G6–G7:** Full marks require modules 1, 2, 4 working well + basic Module 3 + n8n
- **G8 THETA:** Full marks require modules 1 and 2 working + basic AI chat + Workflow 1 (n8n)

Bonus Points

Bonus	Points
Cloud deployment (accessible via public URL)	+5
Grafana monitoring dashboard	+5
Flutter app on a physical Android/iOS device	+3
Unit test coverage ≥ 60% (backend)	+3
PWA (web app installs as app on mobile)	+2
All 5 n8n workflows completed	+2

12. Post-Internship Integration Plan

After all 8 groups submit, CIXIO will run a **4-week integration review**:

Step 1: Code Audit (Week 1 after submission)

CIXIO engineers review all 8 repos. Score each module independently. Identify:

- Best backend API design (schema, auth, code quality)
- Best RAG implementation (chunking strategy, retrieval quality)
- Best Flutter app (architecture, state management, UI quality)
- Best n8n workflows (logic, reusability)
- Best CI/CD pipeline

Step 2: Integration Shortlist (Week 2)

Pick top 2–3 implementations per module. Request permission from interns to integrate their code into CIXIO (with credit in a `CONTRIBUTORS.md`).

Step 3: Port to CIXIO Stack (Weeks 3–4)

- **n8n workflows:** Import directly into CIXIO's n8n instance (CIXIO updates credentials)
- **Flutter mobile screens:** Adapt UI patterns into `cixio-mobile-app`
- **RAG pipeline:** Benchmark against `cixio-ai` — if a group's chunking or retrieval is better, adopt it
- **GitHub Actions templates:** Standardize across CIXIO repos
- **Docker Compose patterns:** Improve CIXIO's `docker-compose.yml` files

What Interns Get

- Written feedback per module
 - CIXIO letter of recommendation for top performers (G1–G3 who achieve full completion)
 - Credit in CIXIO CONTRIBUTORS.md if code is integrated
 - Interview opportunity for top 3–5 individuals
-

13. Rules, Boundaries & Data Privacy

What Interns Are NOT Allowed To Do

1. **No access to CIXIO codebase** — Not shared, not referenced, not asked about
2. **No access to CIXIO GitHub Enterprise** — They use their own GitHub accounts only
3. **No access to CIXIO AWS, GCP, or Azure accounts** — Cloud deploy is on free platforms or CIXIO's intern-only isolated instance
4. **No storing CIXIO customer data** — Do not use any real company data. Use dummy/generated test data only
5. **No committing secrets** — API keys, passwords, tokens must NEVER be committed. Use `.env` files excluded via `.gitignore`
6. **No sharing CIXIO's n8n instance credentials** with outside parties

What Interns ARE Allowed To Do

1. Use publicly available LLM APIs (OpenAI free tier, Google Gemini free tier, Ollama locally)
2. Use free-tier cloud platforms (Railway, Render, Fly.io, Supabase free tier)
3. Use any open-source libraries from npm, PyPI, pub.dev
4. Reference CIXIO's public-facing products (`cixio.ai`, `cixio.dev`) for inspiration — but not for reverse engineering
5. Ask CIXIO contact (designated intern mentor) technical questions — but the mentor will not write code for them

Data Privacy

- All test data must be **fake/generated** (use tools like Faker.js or Python Faker library)
- No real names, real emails, real phone numbers as test data
- If experimenting with RAG: use publicly available documents (Wikipedia articles, public research papers) — not company documents
- CIXIO will **not share any internal documents** with interns for their RAG systems

Intellectual Property

- Code written by interns during this internship on their own machines and their own GitHub accounts **belongs to the interns**
 - If CIXIO wishes to integrate any code, they will request explicit written permission from the intern
 - CIXIO will credit contributors in `CONTRIBUTORS.md`
-

14. Group-Specific Guidance

G1 · ALPHA — Guidance

You are expected to build a production-quality application. Push yourselves:

- Aim for cloud deployment on Fly.io or Railway by Week 6
- Build the monitoring dashboard — Prometheus + Grafana in Docker Compose
- G1's output will be used as the **reference implementation** that CIXIO uses to calibrate evaluations of other groups
- DevOps lead (Dany): create a clean multi-stage Dockerfile and a GitHub Actions pipeline that includes linting, testing, Docker build, and a deploy step
- Challenge: try adding WebSocket support to the AI chat (streaming responses)

G2 · BETA — Guidance

Strong technical group with good AI coverage:

- Both AI/LLM students should split: ABEL owns the chat completion flow, SARFARAS owns the RAG pipeline
- ABHINAND: focus on a clean Docker Compose and a GitHub Actions pipeline. Add Grafana to stretch
- Khushi: build a polished, responsive React frontend — this is where G2 can shine visually

G3 · GAMMA — Guidance

This group has solid backend depth:

- Three backend students should divide clearly: auth+users module (Neeta), notes+tasks module (Aayisha), file upload module (Nandana)

- Raehaan (DevOps): ensure CI pipeline is clean and Docker Compose runs all modules
- A V Kiran: Flutter mobile is your stretch challenge — start with basics and build up

G4 · DELTA — Guidance

Group needs clear coordination to avoid overlap:

- Sahal and Aleena split the backend: Sahal owns auth/users/health, Aleena owns notes/tasks/documents
- Three frontend students: Sanas handles dashboard layout, Achuth handles forms and modals, Akshith leads mobile
- Akarsh: write the CI pipeline and Docker Compose first — this unblocks everyone else
- Amith: start AI module in Week 2. Use OpenAI free tier if Ollama setup takes too long

G5 · EPSILON — Guidance

Three AI/LLM students is the key strength here — divide AI work well:

- Akshay: owns the main chat completion endpoint and system prompt management
- Ajith: owns the RAG ingestion pipeline (file → chunks → embeddings → store)
- Nezil: owns the RAG query pipeline (question → embed → search → answer → citations)
- This group should produce the most thorough AI module of all 8 groups

G6 · ZETA — Guidance

Four backend students is a strength but needs careful coordination:

- Mohammed Kutty: API lead — defines all endpoint contracts first (write OpenAPI spec in Week 1)
- Seljo: database migrations, schema, CRUD
- Aditya: file upload, storage
- Ameen: AI engine (you are the solo AI developer — focus on basic chat + simple RAG)
- Sruthin: Mobile lead — use the Flutter starter templates to get moving quickly

G7 · ETA — Guidance

Three backend and three frontend students — strong balance, but AI is one student:

- Hrideesh: AI module — start with basic chat using Gemini free tier. RAG is stretch
- Three backend students: Amrah leads (auth), Mohammed Roshan (notes/tasks CRUD), Jasim (file upload + documents)
- Three frontend students: Adithyadev (overall layout + routing), Anish (AI chat component), Karthik (Mobile App lead)

- Focus on clean integration between modules in Week 4 — this group's strength is having enough people to work in parallel

G8 · THETA — Guidance (Intensive Mentoring Track)

This group receives more support. The expectations are calibrated lower — focus on quality over quantity:

- **Week 1–2 Priority:** Backend API (auth + notes CRUD) must be working perfectly before moving to anything else
 - **Week 3 Priority:** Connect web frontend to API. Start with just login + notes list
 - **Week 4 Priority:** Get basic AI chat working (use OpenAI free tier — easiest setup)
 - **Week 5 Priority:** n8n Workflow 1 (Welcome Email) working
 - Mobile, RAG, and advanced features are bonus — don't rush them if core modules are not solid
 - CIXIO will schedule additional 30-minute check-in calls every week for this group
 - Honest message: finishing 3 modules well is better than 6 modules half-working
-

Appendix A: Suggested Repository Structure

None

```
smarthub-[groupname]/
├── backend/                                # Module 1 - FastAPI or Node.js
│   ├── app/
│   │   ├── main.py (or index.js)
│   │   ├── routes/
│   │   ├── models/
│   │   ├── database.py
│   │   └── auth.py
│   ├── tests/
│   ├── Dockerfile
│   ├── requirements.txt (or package.json)
│   └── .env.example
├── frontend/                              # Module 2 - React/Next.js
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── api/                          # API client functions
│   │   └── hooks/
│   ├── public/
│   └── Dockerfile
```

```

|   └─ package.json
└─ mobile/                                # Module 3 - Flutter
|   └─ lib/
|       └─ main.dart
|       └─ screens/
|       └─ widgets/
|       └─ services/                    # API calls
|       └─ pubspec.yaml
└─ ai-engine/                            # Module 4 - AI/LLM service
|   └─ app/
|       └─ main.py
|       └─ routes/
|           └─ chat.py
|           └─ rag.py
|       └─ rag/
|           └─ ingester.py
|           └─ embedder.py
|           └─ retriever.py
|       └─ llm/
|           └─ provider.py
|   └─ tests/
|   └─ Dockerfile
|   └─ requirements.txt
└─ automation/                          # Module 5 - n8n workflows
|   └─ workflow-welcome-email.json
|   └─ workflow-daily-digest.json
|   └─ workflow-ai-report.json
|   └─ README.md
└─ .github/
|   └─ workflows/
|       └─ ci.yml                      # Module 6 - GitHub Actions
└─ monitoring/                          # Module 7 - Grafana dashboards
|   └─ prometheus.yml
|   └─ grafana-dashboard.json
└─ docker-compose.yml                  # Runs everything
└─ .env.example                        # Template for environment
variables
└─ README.md                          # Complete project documentation

```

Appendix B: Quick Start Guide for Interns

Step 1: Set up your development environment

None

```
# Required installs on local machine:  
# - Python 3.11+ (python.org)  
# - Node.js 20+ (nodejs.org)  
# - Flutter 3.x (flutter.dev)  
# - Docker Desktop (docker.com)  
# - Git (git-scm.com)  
# - VS Code (code.visualstudio.com)
```

```
# Verify installations:  
python --version  
node --version  
flutter --version  
docker --version
```

Step 2: Create your group's GitHub repository

None

```
# One person creates the repo, others fork or are added as  
# collaborators  
git clone https://github.com/[your-username]/smarthub-[groupname]  
cd smarthub-[groupname]  
  
# Create the folder structure  
mkdir backend frontend mobile ai-engine automation  
.github/workflows monitoring
```

Step 3: Start with the Backend (Week 1)

None

```
# Python FastAPI starter  
cd backend  
python -m venv venv  
source venv/bin/activate # Windows: venv\Scripts\activate
```

```
pip install fastapi uvicorn sqlalchemy python-jose passlib  
python-multipart
```

```
# Create your first file: app/main.py  
# Run: uvicorn app.main:app --reload  
# Open: http://localhost:8000/docs
```

Step 4: Connect to CIXIO's n8n

1. Receive n8n URL and login credentials from CIXIO mentor
 2. Login to n8n dashboard
 3. Create a new workflow
 4. Use the **Webhook** trigger node — copy the webhook URL
 5. In your backend, call that webhook URL when a new user registers
 6. Test by hitting your registration API endpoint
 7. Export workflow as JSON → save to `/automation/` folder
-

Appendix C: Recommended Learning Resources

Backend (FastAPI)

- FastAPI official docs: <https://fastapi.tiangolo.com/>
- SQLAlchemy docs: <https://docs.sqlalchemy.org/>

Frontend (React)

- React docs: <https://react.dev/>
- Tailwind CSS: <https://tailwindcss.com/docs>

Mobile (Flutter)

- Flutter docs: <https://docs.flutter.dev/>
- Dart language tour: <https://dart.dev/guides/language/language-tour>

AI / RAG

- ChromaDB docs: <https://docs.trychroma.com/>
- Ollama: <https://ollama.com/>
- LangChain docs: <https://python.langchain.com/>

n8n

- n8n docs: <https://docs.n8n.io/>
- n8n workflow examples: <https://n8n.io/workflows/>

Docker

- Docker Compose docs: <https://docs.docker.com/compose/>
- Dockerfile best practices:
https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

GitHub Actions

- GitHub Actions docs: <https://docs.github.com/en/actions>

Document prepared for CIXIO — TKM Internship Programme 2026
Questions: Contact your designated CIXIO intern mentor